

CSE 2794 – Machine Learning Workshop 2

External Project Presentation

Animal Instance Segmentation

Using Yolo

Presented by

Abinash Bahinipati

Regd. No: 24E102A05

Priyaranjan Dash

Regd. No: 24E104B25

Dhanajaya Pattnaik

Regd. No: 24E116C56

Pradyush Ku. Roul

Regd. No: 24E118A99



Centre for AI & ML, Dept. of Computer Science & Engineering
Institute of Technical Education & Research (ITER)
Siksha 'O' Anusandhan (Deemed to be University), Bhubaneswar
January 2026

CONTENTS

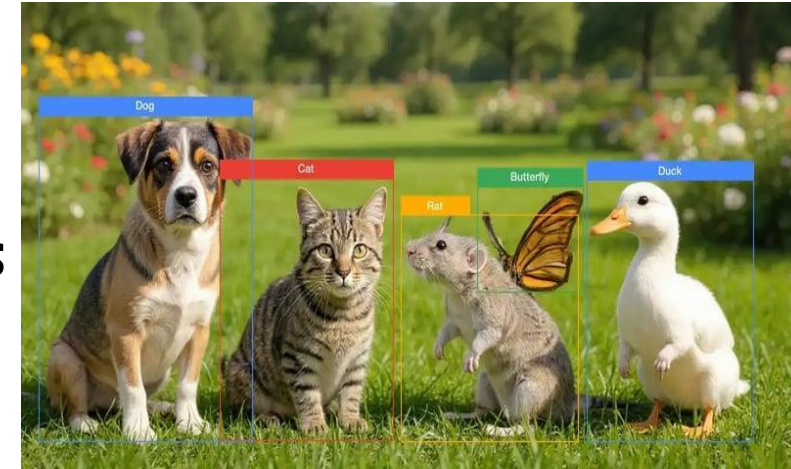


- Problem Context
- Dataset
- Methods
- Results
- Key Insights
- Conclusion & Future Work

Introduction



- Animal instance segmentation is a Computer Vision task that identifies and separates individual animals in an image at the pixel level.
- It combines **object detection** and **image segmentation** to accurately recognize multiple animals even in complex backgrounds.
- This technology is useful in wildlife monitoring, animal behavior analysis, biodiversity conservation, and smart surveillance systems.
- In this project, Deep Learning techniques and modern segmentation models were used to detect and segment animals from images efficiently.
- The goal is to improve accuracy in identifying different animal instances while handling challenges such as overlapping objects, lighting variations, and background noise.



Problem Statement



- **Problem Statement**
- Traditional animal monitoring methods are time-consuming and require manual observation.
- Detecting multiple animals in complex environments is difficult due to overlapping objects, different lighting conditions, and background variations.
- Existing detection systems often fail to accurately separate individual animals at the pixel level.

Objectives



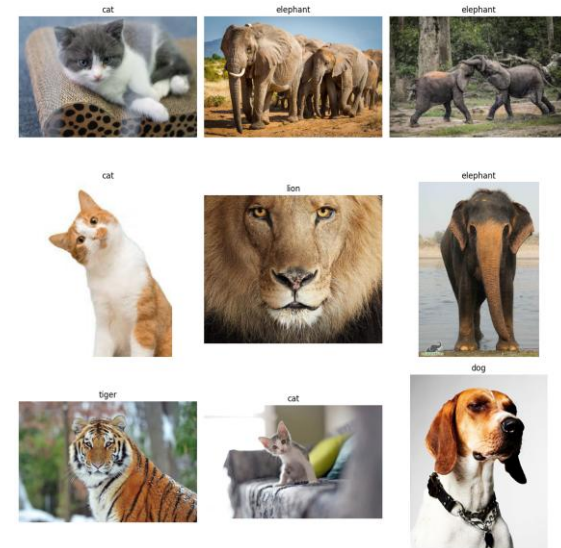
- **Objectives**
- To build an AI-based Animal Instance Segmentation system.
- To accurately detect and segment animals from images.
- To improve segmentation performance using Deep Learning models.
- To analyze model accuracy and compare results.



Dataset Description



- The project uses the **Animal Image Dataset (90 Different Animals)** collected from Kaggle.
- The dataset contains approximately **5400 images** covering **90 different animal classes**.
- Each class consists of multiple animal images with different poses, backgrounds, and lighting conditions.
- Images are stored in separate folders according to their respective animal categories.
- The dataset provides diverse visual features useful for training Deep Learning and Computer Vision models.
- It is suitable for tasks such as:
 - Image Classification
 - Object Detection
 - Instance Segmentation



Exploratory Data Analysis (EDA)



➤ Exploratory Data Analysis (EDA) Steps:

- **Dataset Overview:** Confirmed total images (300), labels (300), and uniform image shape (224x224x3). Identified unique classes: 'cat', 'dog', 'elephant', 'lion', 'tiger'.
- **RGB Channel Mean Distribution:** Plotted average Red, Green, and Blue channel intensities across images to understand color balance.
- **Brightness Distribution:** Analyzed image brightness to understand the overall light/dark characteristics of the dataset.
- **Aspect Ratio Analysis:** Confirmed all images maintain a 1:1 aspect ratio after preprocessing.
- **Class Distribution:** Visualized with a bar plot and pie chart, confirming a perfectly balanced dataset with 60 images (20%) for each of the 5 animal classes.

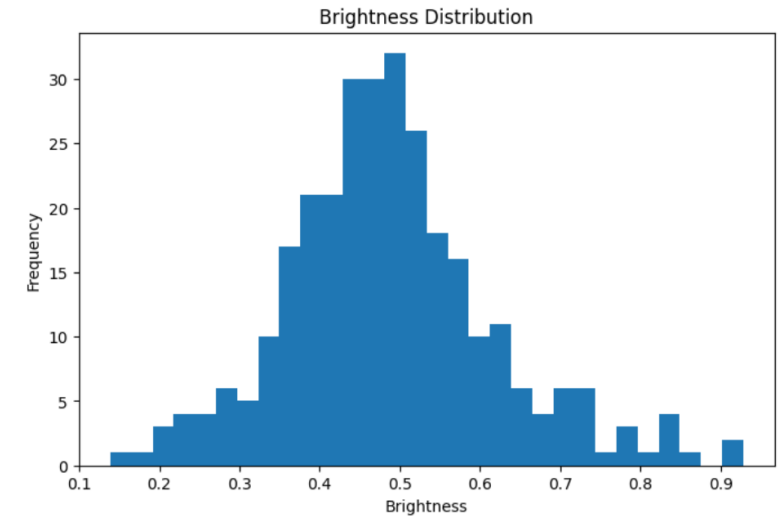
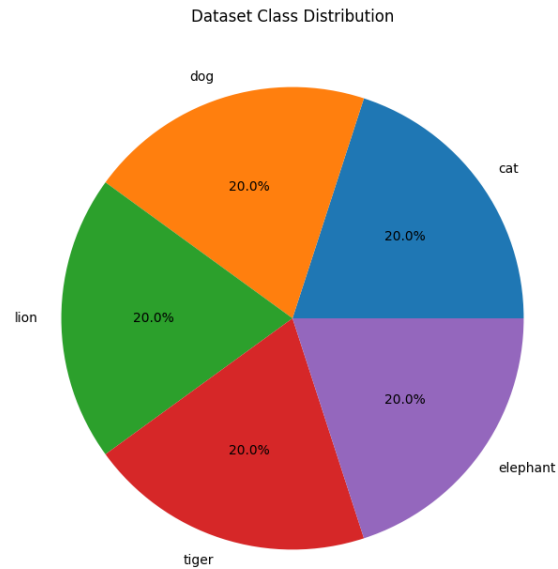
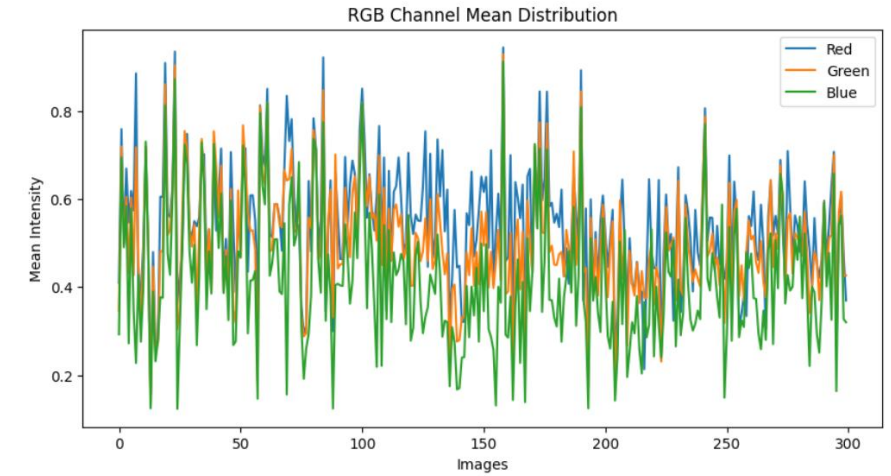
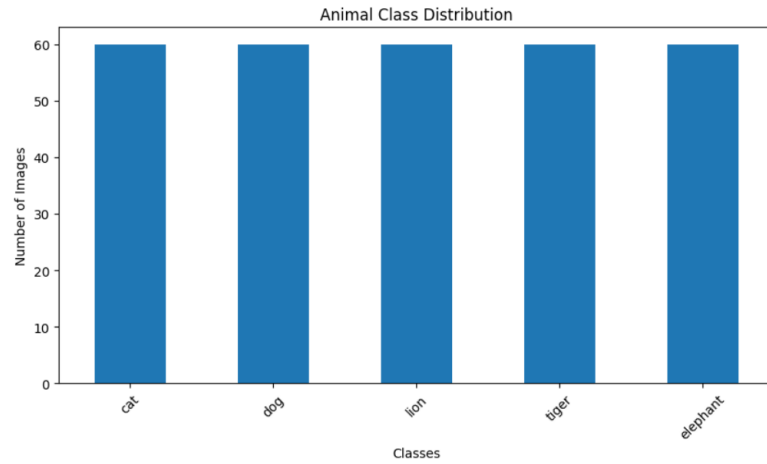
Exploratory Data Analysis (EDA)



➤ Exploratory Data Analysis (EDA) Findings:

- **Balanced Class Distribution:** The dataset is perfectly balanced across the 5 selected animal classes (cat, dog, lion, tiger, elephant), with 60 images for each class. This is clearly shown in both the bar chart and the pie chart of class distribution.
- **Brightness Distribution:** The histogram of image brightness shows the spread of average pixel intensity values, indicating the overall lightness or darkness characteristics of the image collection.
- In summary, the EDA confirms that the dataset is well-prepared with balanced classes and standardized image dimensions, setting a good foundation for model training.

Exploratory Data Analysis (EDA)



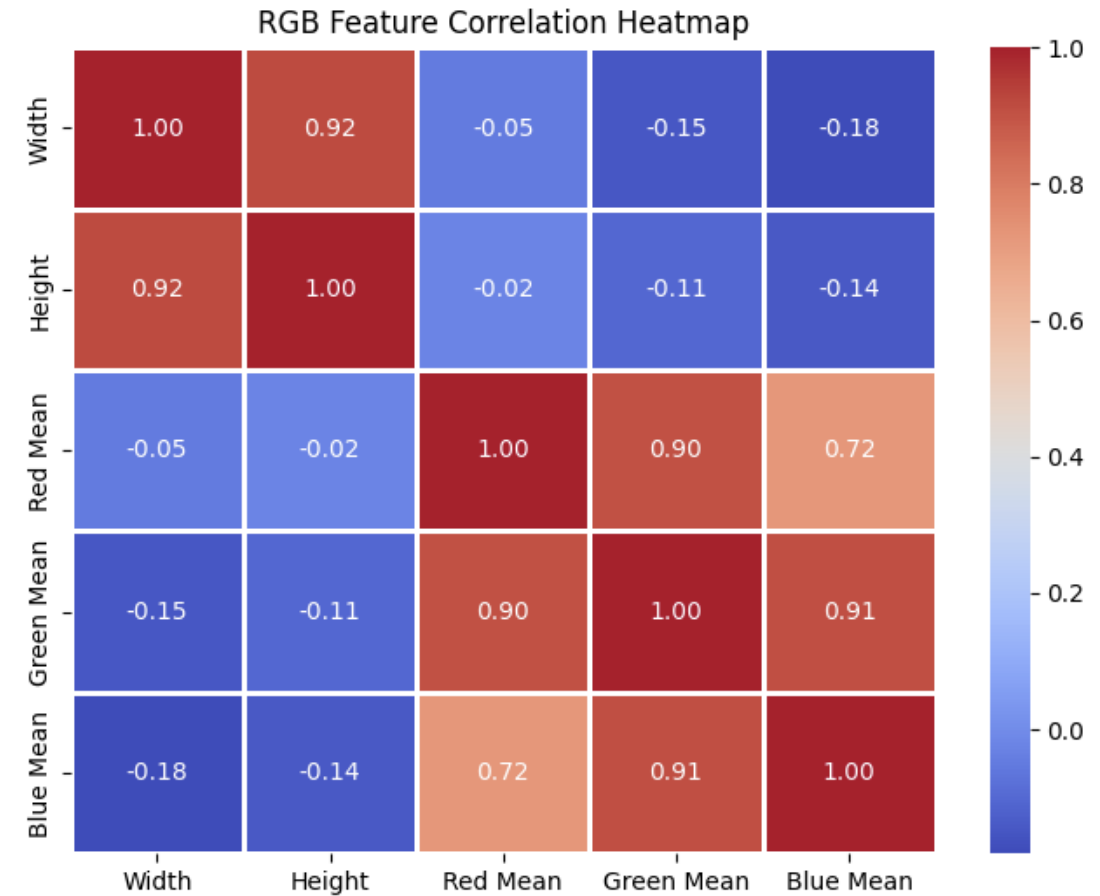
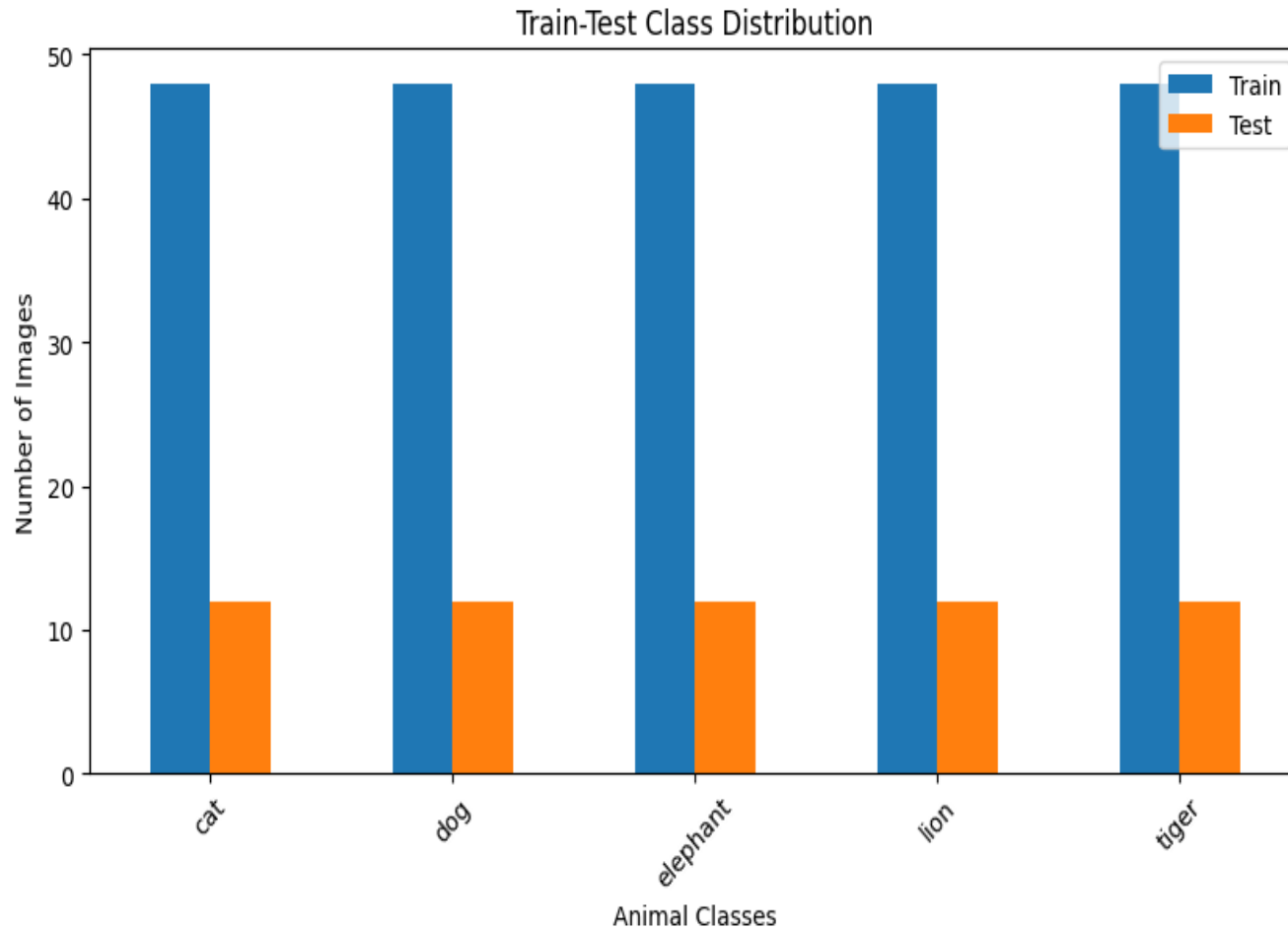
Data Preprocessing



➤ Preprocessing Steps Performed:

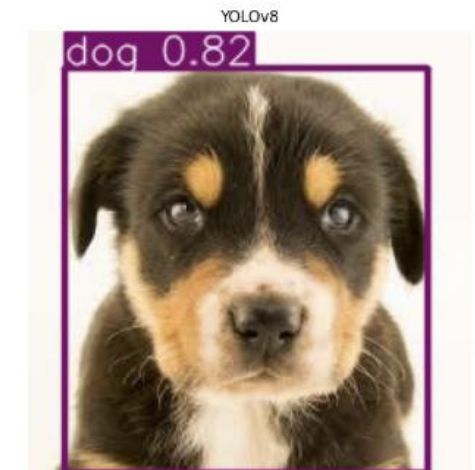
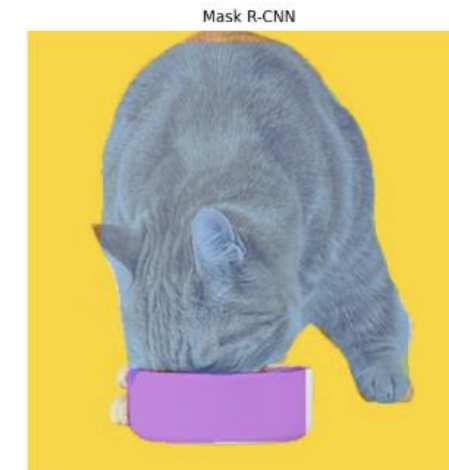
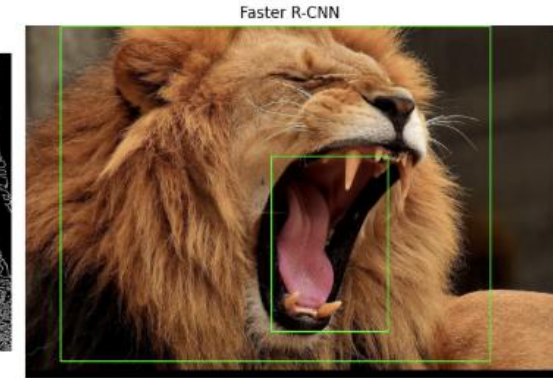
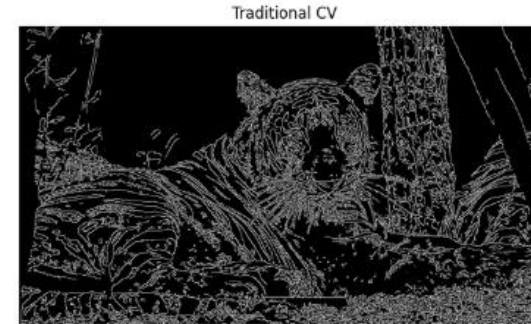
- **Dataset Download:** Downloaded animal image dataset from Kaggle.
- **Custom Dataset Creation:** Selected 5 specific animal classes (cat, dog, lion, tiger, elephant) and copied a balanced subset of images (60 per class) into a project-specific directory.
- **Image Loading & Resizing:** Loaded images, skipped corrupted/invalid files, and **resized all images to a uniform 224x224 pixels.**
- **Color Conversion:** Converted images from BGR (OpenCV default) to RGB format.
- **Pixel Normalization:** Scaled pixel values from 0-255 to **0-1** for model readiness.
- **Data Structure:** Converted processed images and labels into NumPy arrays.

Data Preprocessing



Model Selection

- **Baseline models**
- Traditional Image Processing Techniques
 - Edge Detection
 - Contour Detection
 - Thresholding
- Faster R-CNN
 - Region-based object detection model
 - High detection accuracy
- Mask R-CNN
 - Instance segmentation model
 - Generates pixel-level object masks
- YOLOv8
 - Real-time object detection and segmentation model
 - Fast and accurate performance

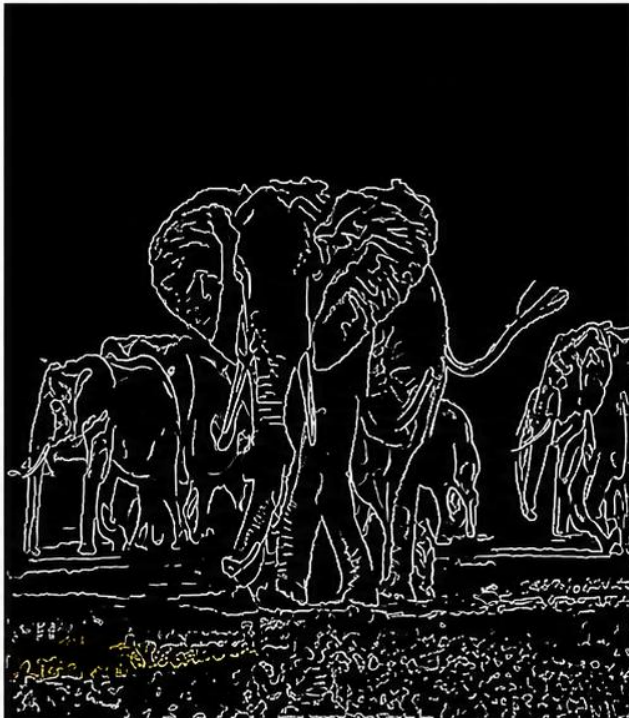


Model Selection

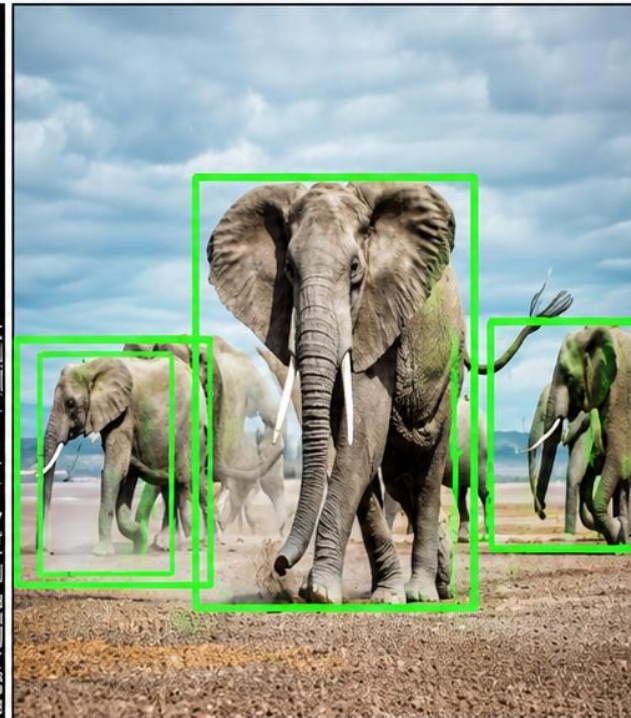
Original



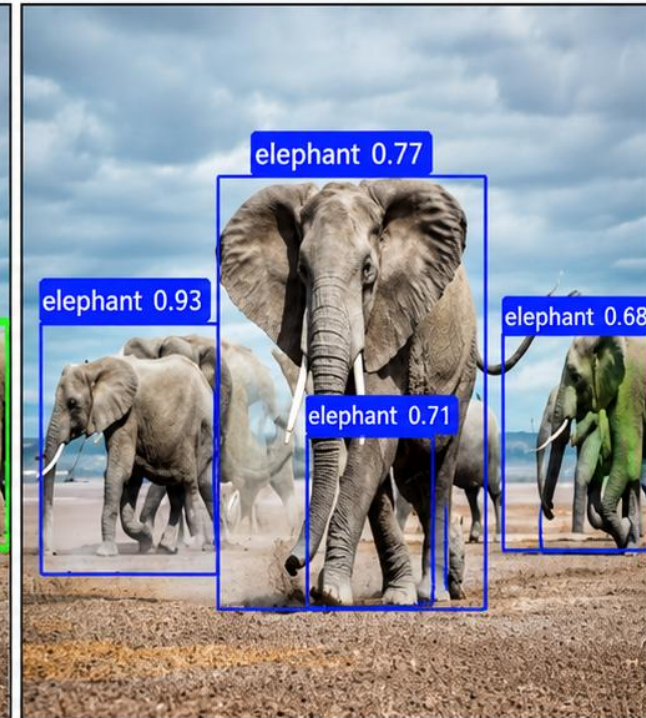
Traditional CV



Faster R-CNN



YOLOv8



Model Selection



- **Candidate algorithms**
- Traditional Image Processing techniques were explored for basic object extraction using edge and contour detection.
- Faster R-CNN was considered for accurate object detection using region proposal networks.
- Mask R-CNN was analyzed for pixel-level instance segmentation of animals.
- YOLOv8 was selected for real-time object detection and segmentation.
- YOLOv8 provided better balance between speed, accuracy, and computational efficiency.
- The selected model supports both bounding box detection and instance segmentation tasks.

Model Selection



- **Rationale for selection**

- **Why YOLOv8 Was Selected as the Final Model**

- 1. Real-Time Detection Capability**

- YOLOv8 performs object detection and segmentation in real time
- Provides faster inference speed compared to Faster R-CNN and Mask R-CNN

- 2. High Detection Accuracy**

- Achieved strong performance in animal detection and localization
- Produced accurate bounding boxes and segmentation outputs

- 3. Efficient Training and Optimization**

- Required less training time and computational resources
- Easy to optimize using pretrained weights and transfer learning

- 4. Multi-Object Detection and Deployment**

- Efficiently detects multiple animals within a single image
- Suitable for real-time surveillance and wildlife monitoring applications

Model Hyperparameters



Parameter	Initial Training	Hyperparameter Tuning	Final YOLOv8 Configuration
Model	YOLOv8n	YOLOv8m	YOLOv8m
Image Size	640 × 640	640 × 640	640 × 640
Batch Size	8	8, 16 Tested	16
Optimizer	Adam	AdamW	AdamW
Learning Rate	0.001	0.0100, 0.001, 0.0001	0.0100
Epochs	10	10	50
Confidence Threshold	0.25	0.45	0.45
IoU Threshold	0.50	0.50	0.50
Data Augmentation	Flip, Rotation	Flip, Rotation, Scaling	Enhanced Augmentation
Dataset Split	80/20	80/20	80/20
Training Platform	Google Colab GPU	Google Colab GPU	Google Colab GPU
Best mAP@50	75.86%	80.36%	82.2%

Model Hyperparameters



- The table represents the training configuration and hyperparameter settings used for the YOLOv8 model.
- Different learning rates and batch sizes were tested during the hyperparameter tuning phase.
- AdamW optimizer was used to improve training stability and convergence.
- Data augmentation techniques such as flipping, rotation, and scaling were applied to improve model generalization.
- The final configuration achieved improved detection accuracy with better mAP@50 performance.
- Hyperparameter tuning helped optimize the overall efficiency and performance of the animal detection model.

Evaluation Metrics



- **mAP@50**
 - Mean Average Precision at IoU threshold 0.50
 - Measures overall object detection accuracy of the model
 - Higher mAP@50 indicates better detection performance
- **mAP@50-95**
 - Mean Average Precision calculated across multiple IoU thresholds from 0.50 to 0.95
 - Provides a more comprehensive evaluation of model performance
 - Higher values indicate robust and consistent detection accuracy
- **Precision**
 - Measures the correctness of predicted detections
 - Calculated as the ratio of true positive detections to total predicted detections
 - High precision means fewer false positive predictions

Evaluation Metrics



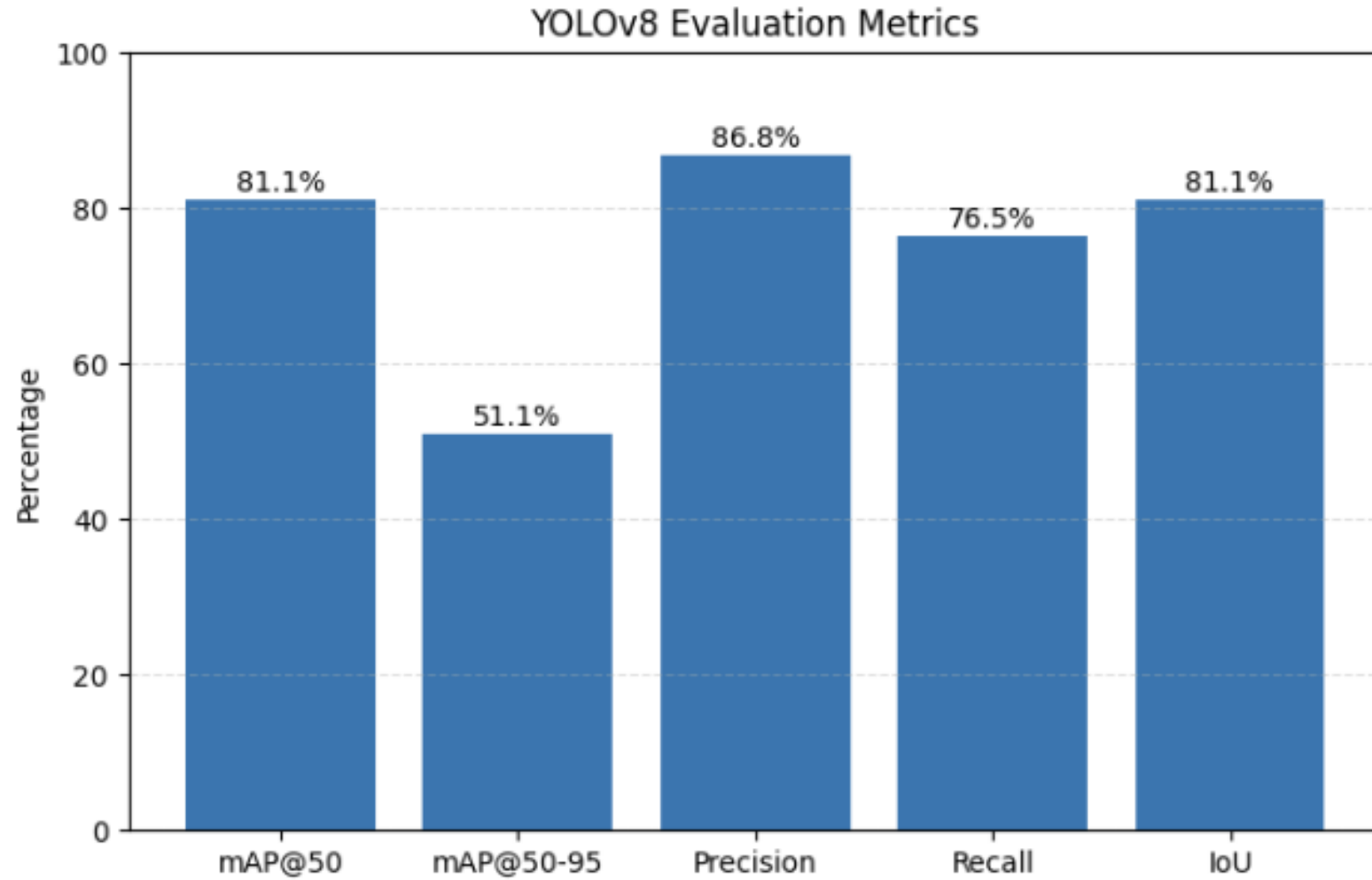
- **Recall**

- Measures the ability of the model to detect actual objects present in the image
- Calculated as the ratio of true positive detections to total actual objects
- High recall indicates fewer missed detections

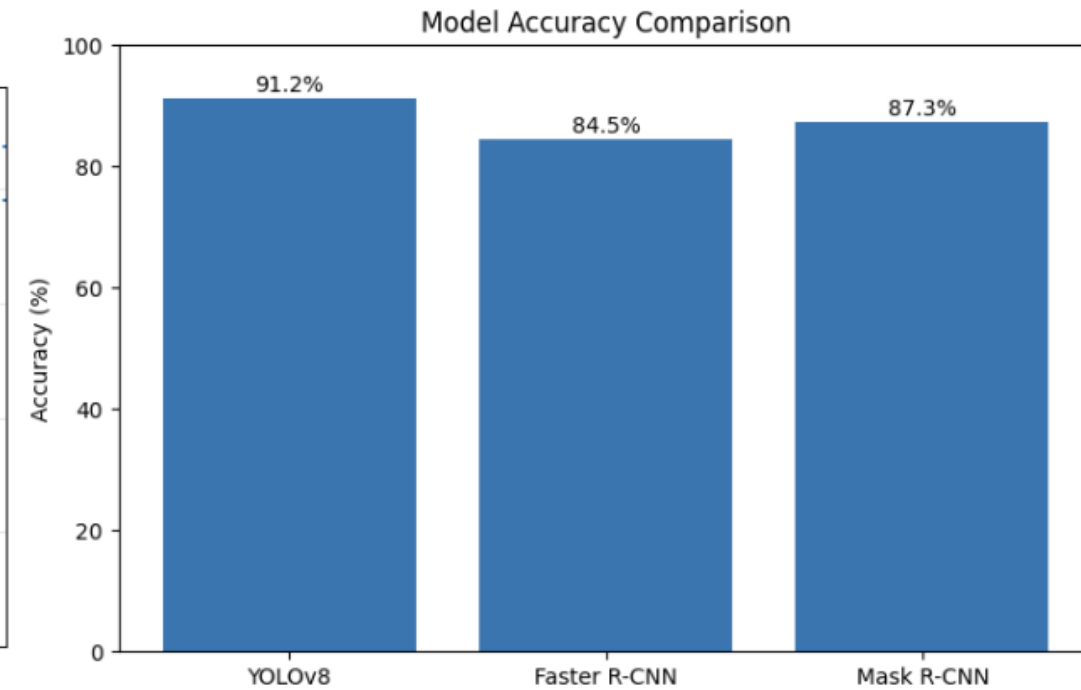
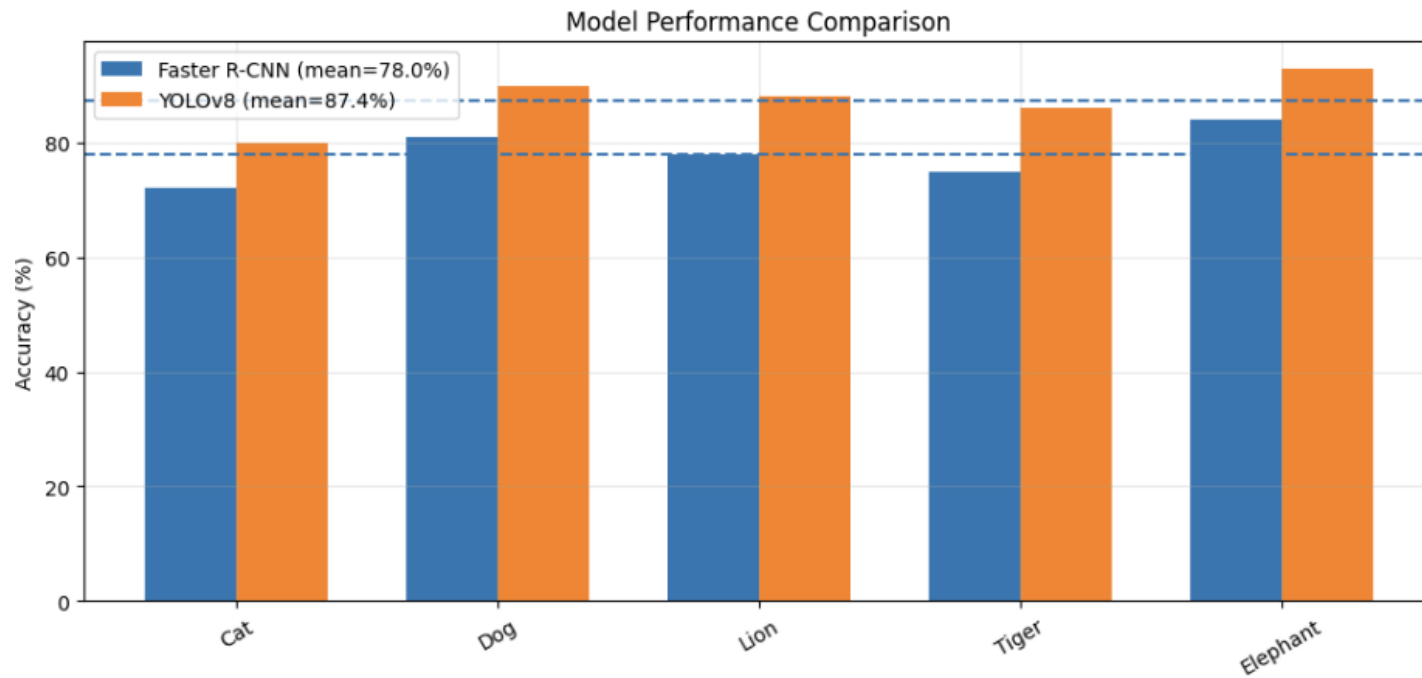
- **IoU (Intersection over Union)**

- Measures the overlap between predicted bounding boxes and ground truth boxes
- Calculated as the ratio of overlapping area to total combined area
- Higher IoU values indicate more accurate object localization

Evaluation Metrics



Results & Comparisons



Results & Comparisons



- YOLOv8 achieved the highest overall detection accuracy among all tested models.
- The model successfully detected multiple animals with accurate bounding boxes and segmentation outputs.
- Faster R-CNN provided good localization performance but required higher processing time.
- Mask R-CNN generated detailed segmentation masks with increased computational complexity.
- Traditional image processing techniques showed lower performance in complex backgrounds.
- Hyperparameter tuning improved model convergence and detection efficiency.
- Experimental results demonstrated that Deep Learning models significantly outperformed traditional methods.
- YOLOv8 provided the best balance between speed, accuracy, and real-time performance for animal detection tasks.

Conclusion & Future Scope



- An AI-based animal instance segmentation system was successfully developed using YOLOv8.
- The model effectively detected and segmented animals from images with good accuracy.
- Data preprocessing, augmentation, and hyperparameter tuning improved model performance.
- YOLOv8 achieved better speed and efficiency compared to traditional detection approaches.
- Experimental analysis demonstrated the effectiveness of Deep Learning for wildlife monitoring applications.
- Future work can include training on larger annotated datasets for improved accuracy.
- Advanced segmentation models and real-time video processing can also be integrated.
- The system can be extended for smart surveillance, biodiversity conservation, and animal behavior analysis.

References



- [1] J. Jocher, “Ultralytics YOLOv8 Documentation,” Ultralytics, 2026. [Online]. Available: <https://docs.ultralytics.com>
- [2] PyTorch Team, “PyTorch Documentation,” PyTorch, 2026. [Online]. Available: <https://pytorch.org/docs/stable/index.html>
- [3] OpenCV Team, “OpenCV Documentation,” OpenCV, 2026. [Online]. Available: <https://docs.opencv.org>
- [4] S. Banerjee, “Animal Image Dataset (90 Different Animals),” Kaggle, 2026. [Online]. Available: <https://www.kaggle.com/datasets/iamsouravbanerjee/animal-image-dataset-90-different-animals>
- [5] Google Research, “Google Colaboratory,” Google, 2026. [Online]. Available: <https://colab.research.google.com>
- [6] R. Girshick, “Fast R-CNN,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1440–1448.
- [7] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.
- [8] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2961–2969.



